

ΜΥΕ041

Διαχείριση Σύνθετων Δεδομένων

Διδάσκων: Ν. Μαμουλής

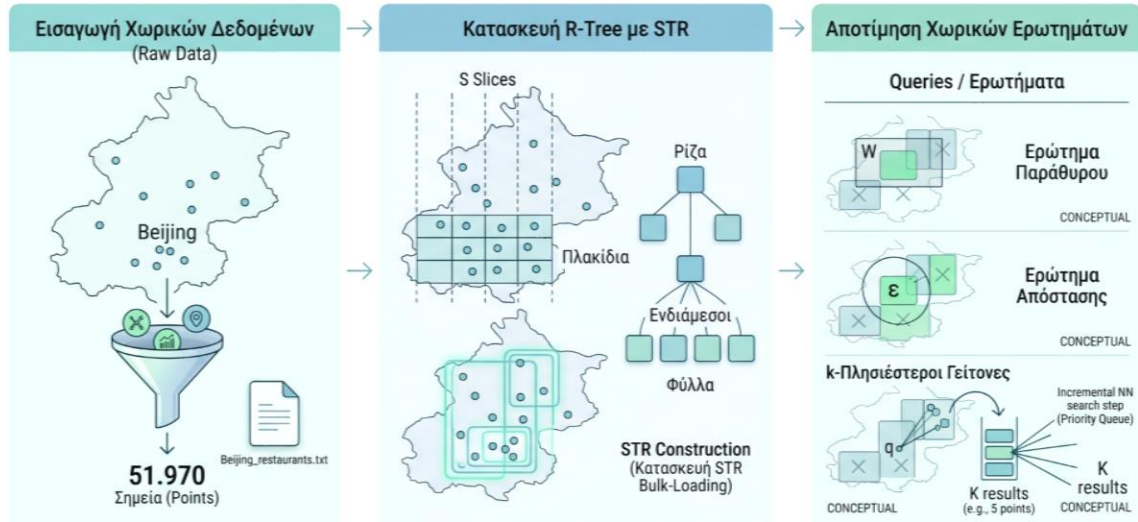
Εργασία 2 - Χωρικά Δεδομένα

Ομάδα:

Παναγιώτης Παρασκευόπουλος ΑΜ: 2905

Άγγελος Μπεζαΐτης ΑΜ: 4432

Ολοκληρωμένο Σύστημα R-Tree: Από τα Δεδομένα στα Ερωτήματα



Περιεχόμενα

Εισαγωγή.....	2
Κατασκευή του R-tree στη Βασική Μνήμη (Μέρος 1)	3
Βασικές Δομές Δεδομένων.....	3
Ο Αλγόριθμος STR (Sort-Tile-Recursive) για τα Φύλλα	4
Διαχείριση Χωρητικότητας και Underflow.....	4
Δυναμική Κατασκευή Επιπέδων και Επιλογή Αλγορίθμου	5
Αποτίμηση Χωρικών Ερωτημάτων (Μέρος 2).....	6
Φόρτωση και Ανακατασκευή - Συνάρτηση <code>load_tree</code>	6
Ερωτήσεις Εύρους (Window & Distance Range Queries)	7
k-Πλησιέστεροι Γείτονες (kNN) - Αλγόριθμος Best-First Search.....	7
Οδηγίες Εκτέλεσης (How to Run)	8

Εισαγωγή

Η διαχείριση σύνθετων δεδομένων, και ειδικότερα των χωρικών δεδομένων, αποτελεί έναν από τους πιο κρίσιμους τομείς των σύγχρονων βάσεων δεδομένων. Σε αντίθεση με τα παραδοσιακά δεδομένα που μπορούν να ταξινομηθούν γραμμικά, τα χωρικά δεδομένα (όπως οι συντεταγμένες GPS) είναι πολυδιάστατα και δεν μπορούμε να τα ταξινομήσουμε με τον κλασικό τρόπο. Χρειαζόμαστε ειδικές δομές, όπως τα ευρετήρια, για να βρίσκουμε γρήγορα σημεία που είναι κοντά μας ή μέσα σε μια συγκεκριμένη περιοχή.

Στην παρούσα εργασία, υλοποιήθηκε ένα R-tree χρησιμοποιώντας την τεχνική Sort-Tile-Recursive (STR) για την ομαδοποίηση 51.970 σημείων που αντιπροσωπεύουν εστιατόρια στο Πεκίνο. Το R-tree είναι μια ιεραρχική δομή που βασίζεται στα Ελάχιστα Ορθογώνια Πλαίσια (MBRs). Η βασική φιλοσοφία της δομής είναι η οργάνωση των δεδομένων με τρόπο που να επιτρέπει το "κλάδεμα" (pruning) μεγάλων τμημάτων του χώρου κατά την αναζήτηση, μειώνοντας έτσι δραστικά τις απαιτούμενες προσπελάσεις στη μνήμη.

Ο αλγόριθμος STR που χρησιμοποιήθηκε για την κατασκευή στοχεύει στη δημιουργία ενός δέντρου με την υψηλότερη δυνατή πληρότητα κόμβων και το ελάχιστο δυνατό "νεκρό χώρο" (dead space), διασφαλίζοντας ότι τα ερωτήματα τύπου Window, Distance και k-Nearest Neighbors θα εκτελούνται με τη μέγιστη ταχύτητα.

Κατασκευή του R-tree στη Βασική Μνήμη (Μέρος 1)

Στο πρώτο μέρος της εργασίας υλοποιήθηκε η δομή του R-tree και η φόρτωση των δεδομένων των εστιατορίων (bulk loading). Η αρχιτεκτονική του κώδικα (rtree.py) σχεδιάστηκε με αντικειμενοστραφή λογική (OOP), ώστε να είναι καθαρός, εύκολος στην ανάγνωση και να μπορεί να εμπλουτιστεί εύκολα στο μέλλον.

Βασικές Δομές Δεδομένων

Για την αναπαράσταση των οντοτήτων του χώρου δημιουργήθηκαν τρεις βασικές κλάσεις:

Κλάση **Point**: Αναπαριστά τα εστιατόρια (τα φύλλα του δέντρου) με συντεταγμένες (x, y). Κάθε σημείο έχει ένα μοναδικό record_id, το οποίο αντιστοιχεί στη γραμμή του αρχείου από όπου προήλθε, ώστε να μπορούμε να το αναγνωρίζουμε αμέσως.

Κλάση **MBR** (Minimum Bounding Rectangle): Υλοποιεί τα Ελάχιστα Ορθογώνια Πλαίσια. Κάθε MBR ορίζεται από τις συντεταγμένες της κάτω-αριστερά (x_{low} , y_{low}) και της πάνω-δεξιά (x_{high} , y_{high}) γωνίας του. Περιλαμβάνει επίσης τη συνάρτηση area(), που υπολογίζει αυτόματα το εμβαδό του πλαισίου, κάτι που χρειαζόμαστε για τα τελικά στατιστικά του δέντρου.

Κλάση **Node**: Αποτελεί τον δομικό λίθο του R-tree. Κάθε κόμβος έχει ένα node_id, το οποίο λειτουργεί ως pointer στη διεύθυνση ενός block δίσκου (στην περίπτωση μας, της λίστας rtree_nodes). Επίσης έχει το flag is_leaf (0 για φύλλα, 1 για ενδιάμεσους κόμβους) και τη λίστα entries που περιέχει τα παιδιά του (Σημεία ή MBRs). Η μέθοδος get_mbr() αναλαμβάνει να υπολογίσει δυναμικά το συνολικό MBR που περικλείει όλες τις εγγραφές του κόμβου.

Ο Αλγόριθμος STR (Sort-Tile-Recursive) για τα Φύλλα

Η μέθοδος εξαγωγής και ομαδοποίησης υλοποιήθηκε στη συνάρτηση **build_leaves**. Δεδομένου ότι η χωρητικότητα κάθε block είναι 1024 bytes, ορίστηκε μέγιστη χωρητικότητα $M_L = 51$ σημεία ανά φύλλο.

Για την αποδοτική χωρική ομαδοποίηση των $N = 51.970$ σημείων εφαρμόστηκε ο αλγόριθμος STR:

Χρησιμοποιώντας τη βιβλιοθήκη math, υπολογίζονται δυναμικά το πλήθος των απαιτούμενων φύλλων (P) και οι κάθετες λωρίδες (S) με τις εντολές `math.ceil(N / MAX_LEAF_ENTRIES)` και `math.ceil(math.sqrt(P))`.

Slicing & Tiling:

1. Πραγματοποιείται η αρχική ταξινόμηση ολόκληρης της λίστας των σημείων ως προς τον άξονα X με τη χρήση της `points.sort(key=lambda p: p.x)`.
2. Η λίστα χωρίζεται σε τμήματα μεγέθους `slice_capacity`.
3. Για κάθε τμήμα (λωρίδα), εκτελείται δευτερεύουσα ταξινόμηση ως προς τον άξονα Y (`slice_points.sort(key=lambda p: p.y)`), διασφαλίζοντας τη δημιουργία chunks που είναι χωρικά συμπαγή.

Διαχείριση Χωρητικότητας και Underflow

Ένας θεμελιώδης κανόνας των R-trees είναι ότι κανένας κόμβος, πλην της ρίζας, δεν επιτρέπεται να έχει πληρότητα μικρότερη από το 40% της μέγιστης χωρητικότητάς του. Για τα φύλλα το ελάχιστο όριο είναι $m_L = 20$ και για τους ενδιάμεσους $m_{Int} = 11$. Για να αντιμετωπιστεί το φαινόμενο του Underflow, υλοποιήθηκε με μια "επιθετική" διόρθωση στο τέλος των συναρτήσεων **build_leaves** και **build_level**.

Λογική Δανεισμού: Αν το τελευταίο chunk της λίστας έχει μέγεθος μικρότερο από το MIN_LEAF_ENTRIES, το πρόγραμμα υπολογίζει τον αριθμό των στοιχείων που λείπουν (needed).

List Slicing: Με τη χρήση των ισχυρών εργαλείων slicing της Python, τα στοιχεία μεταφέρονται από τον προτελευταίο κόμβο στον τελευταίο:

```
leaf_chunks[-1] = leaf_chunks[-2][-needed:] + leaf_chunks[-1]
leaf_chunks[-2] = leaf_chunks[-2][: -needed]
```

Αυτός ο τρόπος εγγυάται ότι ο τελευταίος κόμβος θα έχει ακριβώς το ελάχιστο όριο, ενώ ο προηγούμενος θα παραμείνει εντός των νόμιμων ορίων (αφού ξεκίνησε με MAX στοιχεία).

Δυναμική Κατασκευή Επιπέδων και Επιλογή Αλγορίθμου

Η δόμηση του κορμού του δέντρου πραγματοποιήθηκε με τη συνάρτηση **build_rtree**, η οποία εφαρμόζει μια επαναληπτική Bottom-Up διαδικασία. Ξεκινώντας από τους κόμβους-φύλλα (Επίπεδο 0), οι κόμβοι ομαδοποιούνται σε νέα MBRs χωρητικότητας $M_{Int} = 28$ ανά ενδιάμεσο κόμβο. Η αναδρομική αυτή λογική ελέγχεται από έναν βρόχο (while loop), ο οποίος αυξάνει το επίπεδο (level) μέχρι ο αριθμός των παραγόμενων κόμβων να είναι μικρότερος ή ίσος του 28, σηματοδοτώντας τη δημιουργία της Ρίζας (Root).

Για τη διαχείριση αυτής της διαδικασίας, σχεδιάστηκε η συνάρτηση-Wrapper **build_level**. Αυτή επιτρέπει την εναλλαγή μεταξύ δύο προσεγγίσεων μέσω της παραμέτρου USE_BASIC_ALGORITHM:

build_level_basic: Πραγματοποιεί σειριακή ομαδοποίηση των παιδιών-κόμβων. Η επιλογή αυτή διασφαλίζει την απόλυτη ταύτιση της εξόδου με το σύστημα αξιολόγησης (rtree.csv) και τις προδιαγραφές της εκφώνησης.

build_level_optimized: Εφαρμόζει πλήρως τη θεωρία του αλγορίθμου STR και στα ενδιάμεσα επίπεδα. Χρησιμοποιεί τη βοηθητική συνάρτηση `get_center(node)` για την εκ νέου ταξινόμηση των MBRs με βάση το γεωμετρικό τους κέντρο πριν την ομαδοποίηση.

Τέλος, για την προσομοίωση της λειτουργίας του δίσκου, κάθε νέος κόμβος αποθηκεύεται στην καθολική λίστα `rtree_nodes`. Το `node_id` κάθε κόμβου δείχνει την ακριβή του θέση στη λίστα. Με αυτόν τον τρόπο, ο κώδικας λειτουργεί σαν να αναζητά δεδομένα απευθείας σε έναν σκληρό δίσκο.



Δισδιάστατη αναπαράσταση της δομής του R-Tree για τα εστιατόρια του Πεκίνου. Απεικονίζεται η ιεραρχία των Ελάχιστων Περικλειόντων Ορθογωνίων (MBRs) ανά επίπεδο δέντρου, ξεκινώντας από το συνολικό όριο της ρίζας έως τις τελικές συντεταγμένες των σημείων

Αποτίμηση Χωρικών Ερωτημάτων (Μέρος 2)

Στο δεύτερο μέρος της εργασίας, το ενδιαφέρον μετατοπίζεται από την κατασκευή στην ανάκτηση πληροφορίας. Ο κώδικας (**queries.py**) υλοποιεί τρεις θεμελιώδεις τύπους χωρικών αναζητήσεων, χρησιμοποιώντας το R-tree που αποθηκεύτηκε στο αρχείο `rtree.csv`.

Φόρτωση και Ανακατασκευή - Συνάρτηση `load_rtree`

Πριν την εκτέλεση οποιασδήποτε ερώτησης, το δέντρο πρέπει να φορτωθεί στη μνήμη. Η υλοποίηση της **`load_rtree`** επιτυγχάνει τη μετατροπή των δεδομένων κειμένου σε αντικείμενα Python με έναν ιδιαίτερα αποδοτικό τρόπο:

Για Parsing χρησιμοποιήθηκε η βιβλιοθήκη `ast.literal_eval` για τη μετατροπή των εγγραφών από strings σε έτοιμες δομές δεδομένων. Αυτό επιτρέπει την ταχύτατη ανακατασκευή των αντικειμένων Point και MBR.

Με Index-based Mapping, οι κόμβοι τοποθετούνται στη λίστα `rtree_nodes` βάσει του `node_id` τους, εξασφαλίζοντας προσπέλαση σε χρόνο $O(1)$ κατά τη διάρκεια της αναζήτησης.

Ερωτήσεις Εύρους (Window & Distance Range Queries)

Η αποδοτικότητα των ερωτήσεων εύρους βασίζεται στην ικανότητα του R-tree να απορρίπτει ολόκληρους κλάδους του δέντρου που δεν τέμνονται με την περιοχή ενδιαφέροντος.

Window Query: Υλοποιήθηκε στη συνάρτηση **`window_range_query(root_id, W)`**. Χρησιμοποιεί μια βοηθητική δομή (ουρά) για την επαναληπτική διάσχιση του δέντρου. Η συνάρτηση **`check_mbr_intersection`** αποτελεί το κριτήριο για το αν ένας ενδιαμέσος κόμβος πρέπει να εξεταστεί περαιτέρω, ενώ η **`check_point_in_mbr`** εκτελεί τον τελικό έλεγχο στα σημεία των φύλλων.

Distance Query: Η συνάρτηση **`distance_range_query(root_id, q, radius)`** μετατρέπει την κυκλική περιοχή αναζήτησης σε γεωμετρικό κριτήριο. Εδώ, η προγραμματιστική υλοποίηση της MINDIST (**`min_dist_to_mbr`**) είναι καθοριστική: αν η ελάχιστη απόσταση του σημείου ερώτησης q από ένα MBR είναι μεγαλύτερη από την ακτίνα *radius*, τότε ολόκληρος ο κόμβος κλαδεύεται ακαριαία.

k-Πλησιέστεροι Γείτονες (kNN) - Αλγόριθμος Best-First Search

Ο πιο σύνθετος αλγόριθμος της εργασίας υλοποιήθηκε στη συνάρτηση **`knn_query`**. Σε αντίθεση με τα Range Queries, εδώ η διάσχιση δεν είναι τυχαία αλλά κατευθυνόμενη από την εγγύτητα.

Για την ουρά προτεραιότητας (Priority Queue) χρησιμοποιήθηκε η βιβλιοθήκη `heapq` για τη διατήρηση ενός Min-Heap. Η ουρά αποθηκεύει πλειάδες της μορφής (distance, type, id), όπου το dist είναι είτε η MINDIST ενός MBR είτε η ακριβής απόσταση ενός σημείου.

Με το Incremental Search, ο αλγόριθμος εξάγει σταδιακά τα στοιχεία από το Heap. Όταν ένα στοιχείο τύπου “Σημείο” φτάσει στην κορυφή της ουράς, είναι μαθηματικά βέβαιο ότι αποτελεί τον επόμενο πλησιέστερο γείτονα, επιτρέποντας την επιστροφή των αποτελεσμάτων χωρίς την ανάγκη εξέτασης ολόκληρου του δέντρου.

Οδηγίες Εκτέλεσης (How to Run)

Ο κώδικας είναι υλοποιημένος σε Python 3 και χωρίζεται σε δύο κύρια μέρη: την κατασκευή του δέντρου και την εκτέλεση των ερωτημάτων. Τα αρχεία κώδικα (`rtree.py`, `queries.py`) και τα αρχεία δεδομένων (`Beijing_restaurants.txt`, `windowRangeQueries.txt`, `distanceRangeQueries.txt`, `NNQueries.txt`) πρέπει να βρίσκονται στον ίδιο φάκελο.

Μέρος 1: Κατασκευή του R-tree: Για να διαβάσετε τα δεδομένα και να δημιουργήσετε το αρχείο `rtree.csv`, εκτελέστε την παρακάτω εντολή στο τερματικό:

`python rtree.py Beijing_restaurants.txt rtree.csv`

Είσοδος: Το αρχείο με τις συντεταγμένες των εστιατορίων.

Έξοδος: 1. Το αρχείο `rtree.csv` που περιέχει τη δομή του δέντρου. 2. Στατιστικά στην οθόνη (ύψος δέντρου, πλήθος κόμβων ανά επίπεδο και μέσο εμβαδό MBR).

Μέρος 2: Εκτέλεση Χωρικών Ερωτημάτων: Αφού δημιουργηθεί το `rtree.csv`, μπορείτε να εκτελέσετε τα ερωτήματα χρησιμοποιώντας το αρχείο `queries.py`.

A. Ερωτήσεις Εύρους με Παράθυρο (Window Range Queries):

`python queries.py rtree.csv windowRangeQueries.txt`

B. Ερωτήσεις Εύρους με βάση την Απόσταση (Distance Range Queries):

`python queries.py rtree.csv distanceRangeQueries.txt`

Γ. Ερωτήσεις k-Πλησιέστερων Γειτόνων (k-NN Queries):

Για την αναζήτηση των k πλησιέστερων γειτόνων, πρέπει να δώσετε και την τιμή του k ως τρίτο όρισμα (αν δεν δοθεί, η προεπιλεγμένη τιμή είναι 10):

`python queries.py rtree.csv NNQueries.txt 10`